SOUTHEAST UNIVERSITY

Department of Computer Science and Engineering

LAB PROJECT REPORT

SDN Based Automatic Traffic Rerouting and Link Failure Recovery in a Redundant Campus Network

Technology: Software-Defined Networking (SDN)

Student Name	Tanvirul Hoque Faysal
Student ID	Withheld for public copy
Course Code	CSE348.1
Course Title	Advanced Networking LAB
Instructor	Mr. Khaza Newaz Muhammad
Submission Date	23 September 2026

Abstract

This project designs and experimentally evaluates a Software-Defined Networking (SDN) solution for a campus-style network that cannot adapt efficiently to link failure when only a single forwarding path is available. The implementation uses a FAUCET controller, four Open vSwitch (OVS) datapaths, three client hosts, one application server, and a separate management segment in GNS3. The data network operates as VLAN 100 in the 192.168.10.0/24 subnet, while OpenFlow control traffic uses the 10.0.0.0/24 management network. A baseline condition is created by disabling the alternate path, after which failure of the OVS2-to-OVS3 link causes service interruption with no automatic recovery. The proposed configuration enables FAUCET stacking over two physical paths: OVS1-OVS2-OVS3 and OVS1-OVS4-OVS3. Experiments use ICMP and iperf3 to measure packet loss, recovery time, throughput, UDP jitter, and TCP retransmissions. In the measured failure tests, the baseline recorded 44.8718% packet loss and required manual restoration, while the redundant SDN test automatically recovered connectivity with 21.843% loss in the earlier full run; a repeat evidence capture showed an estimated ICMP recovery interval of about 13.2 s. A simultaneous three-service test also demonstrated the different failure effects on UDP video and TCP business/file traffic. The results show that the implemented SDN topology improves resilience by restoring forwarding through an alternate path without manual path reconfiguration.

Keywords: Software-Defined Networking, OpenFlow, FAUCET, Open vSwitch, network resilience, automatic rerouting, link failure, iperf3.

1. Introduction

1.1 Problem Statement and Scenario

The scenario represents a small university or enterprise campus in which three user services reach a central application server through a shared switching fabric. In a basic single-path design, normal communication works, but the network cannot continue carrying traffic when a critical inter-switch link fails. This creates avoidable service interruption and requires manual recovery. The practical problem is therefore not simple reachability; it is the inability of the forwarding fabric to adapt to a changing link state.

The proposed solution applies SDN so that forwarding behavior is centrally controlled and the topology contains a redundant path. The control plane is implemented with FAUCET and the forwarding plane uses Open vSwitch. OpenFlow 1.3 connects the controller to the OVS datapaths. This design follows the SDN principle of separating network control from packet forwarding **[1],** while FAUCET stacking is used to operate the multi-switch topology as a coordinated fabric that can recover from switch or port/cable failures **[3].**

1.2 Objectives and Scope

The project has four practical objectives:

(1) Build an SDN topology containing more than six nodes/devices.

(2) Implement three distinct traffic services.

(3) Create a baseline single-path condition and compare it with the redundant SDN configuration under a link failure.

(4) Collect measurable performance results that can be reproduced during the viva. The project focuses on Layer-2 forwarding and resilience inside one campus subnet. Inter-subnet routing, Internet access, BGP, MPLS, and QoS scheduling are intentionally outside the scope.

2. Technical Background and Design Rationale

2.1 SDN and OpenFlow

The Open Networking Foundation defines SDN around the separation of the control plane from the forwarding plane, with a logically centralized control function managing multiple forwarding devices **[1].** In this project, FAUCET acts as the controller and the four OVS instances act as datapaths. This is appropriate for the problem because the controller can maintain a network-wide view instead of treating each switch as an isolated forwarding device. OpenFlow is used as the southbound control protocol between FAUCET and OVS; the implementation explicitly enables OpenFlow 1.3 on bridge br0. The OpenFlow 1.3.5 specification is an authoritative reference for the switch-controller message model and flow programming behavior **[2].**

2.2 FAUCET Stacking and Open vSwitch

FAUCET supports both independent switching and distributed switching through stacking. Its stacking documentation specifically describes resilient topologies that can recover from switch and port/cable failures **[3].** This matches the project requirement because the physical data plane contains two possible paths between OVS1 and OVS3. The stack root is OVS1, and the inter-switch ports are defined as stack links rather than ordinary access VLAN ports. Open vSwitch is used as the software datapath. The bridge is configured in secure fail mode so that forwarding depends on controller-installed OpenFlow rules instead of silently reverting to uncontrolled standalone switching **[4].**

2.3 Measurement Tools and Transport Protocols

iperf3 is used because it supports TCP and UDP throughput tests and reports interval and summary measurements such as transfer rate, packet loss, and related parameters **[7].** The video service is represented with UDP because UDP provides datagram delivery without guaranteed reliability or retransmission **[6].** Business and file-transfer services use TCP, which provides reliable byte-stream delivery and retransmission mechanisms **[5].** This distinction is useful during failure analysis: UDP loss is directly visible at the receiver, while TCP may show zero-throughput intervals followed by retransmissions and catch-up after connectivity is restored.

2.4 Why No Router Is Used

All application hosts are intentionally placed in the same VLAN and IP subnet (192.168.10.0/24). The experiment therefore studies Layer-2 SDN forwarding and failover rather than inter-subnet routing. A router would be required if the scenario connected different IP networks, branches, or the Internet, but it is not required for this same-subnet resilience experiment. This keeps the implementation aligned with the selected technology, SDN, rather than introducing a second routing problem.

3. Network Design

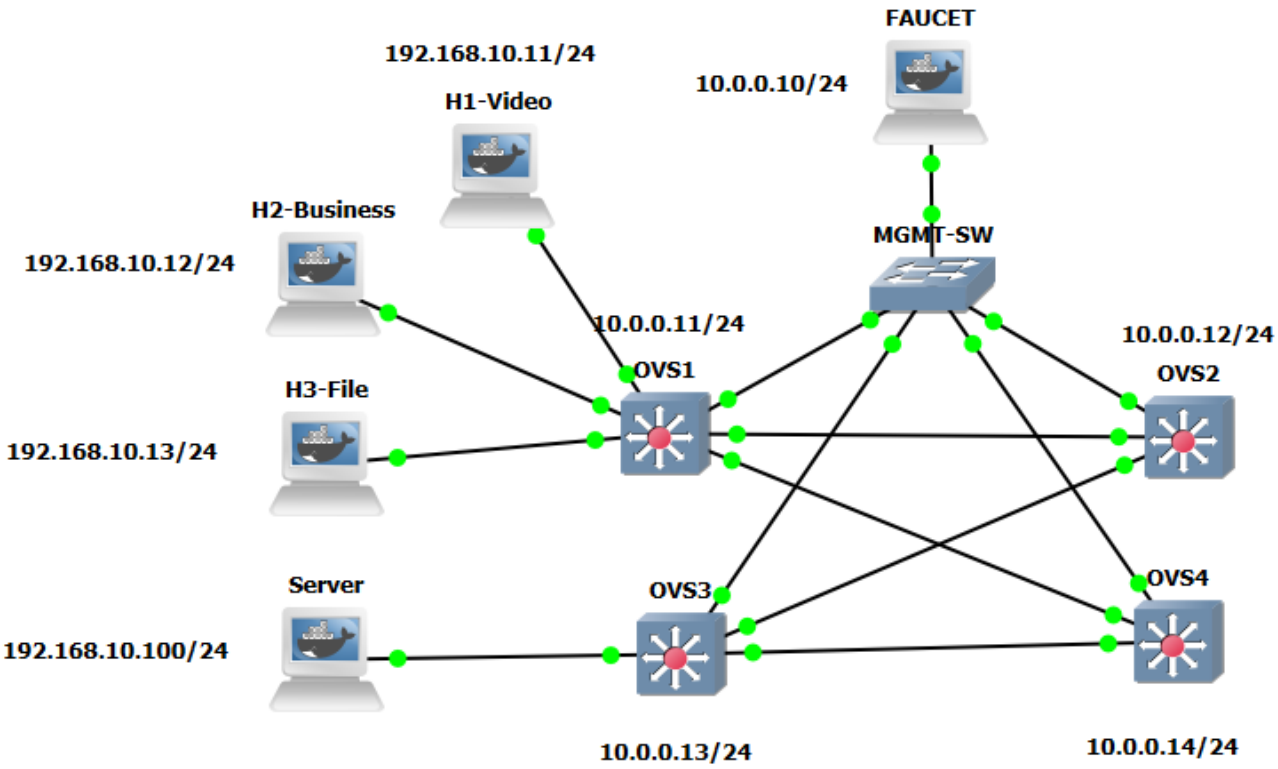


Figure 1. Implemented SDN topology: separate management/control network and redundant data-plane paths.

3.1 Nodes and Roles

Node / Device	Role	Key Address / Function
FAUCET	SDN controller	10.0.0.10/24, OpenFlow TCP 6653
MGMT-SW	Management Ethernet switch	Connects controller and OVS eth0 interfaces
OVS1	Access / stack-root switch	10.0.0.11/24; H1/H2/H3 access; uplinks to OVS2 and OVS4
OVS2	Primary-path transit switch	10.0.0.12/24; OVS1 ↔ OVS3
OVS3	Server-side switch	10.0.0.13/24; server access; links to OVS2 and OVS4
OVS4	Alternate-path transit switch	10.0.0.14/24; OVS1 ↔ OVS3
H1 Video	UDP client	192.168.10.11/24
H2 Business	TCP client	192.168.10.12/24
H3 File	TCP client	192.168.10.13/24
Server	iperf3 application server	192.168.10.100/24

The topology contains ten nodes/devices in total, and even if the controller and management switch are excluded from the count, the four OVS datapaths plus four end hosts still exceed the minimum six-node requirement.

3.2 OpenFlow Port Mapping and Datapath IDs

Switch	Datapath ID	Relevant OpenFlow ports
OVS1	0000e2a24fcdff4a	OF2=H1, OF3=H2, OF4=H3, OF5=OVS2, OF6=OVS4
OVS2	0000daa0cf021f4a	OF2=OVS1, OF3=OVS3
OVS3	00007efabca1aa49	OF2=OVS2, OF3=OVS4, OF4=Server
OVS4	000066bf9a92ef47	OF2=OVS1, OF3=OVS3

3.3 Traffic Services

Flow	Source → Destination	Protocol	Configured rate	Purpose
Video	H1 → Server	UDP	6 Mbit/s	Delay-sensitive media-style traffic
Business	H2 → Server	TCP	3 Mbit/s	Transactional/business application traffic
File	H3 → Server	TCP	8 Mbit/s	Bulk/file-transfer traffic

4. Baseline Configuration: Before the Proposed Resilient Solution

The baseline represents a basic single-path forwarding condition. Host addressing, VLAN 100, OpenFlow control, and normal OVS forwarding remain functional, but the alternate OVS4 path is administratively disabled at OVS1. Traffic therefore depends on OVS1 → OVS2 → OVS3 → Server. This baseline is intentionally simple and provides a controlled before condition for evaluating resilience.

```
OVS1:/# ip link set eth5 down    # disables OVS1 -> OVS4 alternate path
H1:/#   ping -i 0.2 192.168.10.100
OVS2:/# ip link set eth2 down    # injects failure on OVS2 -> OVS3
```

With the alternate path disabled, failure of OVS2 eth2 removes the only complete data path. The expected behavior is continued packet loss until the failed link is manually restored. The baseline therefore demonstrates the limitation being addressed: the network does not have automatic path continuity when the only active transit path fails.

5. Proposed SDN Solution

The proposed solution enables both redundant paths and lets FAUCET manage the multi-switch topology through stacking. OVS1 is configured as the stack root. The inter-switch links are declared as stack links, while H1, H2, H3, and the server are access ports in VLAN 100. The controller programs forwarding rules into the OVS flow tables and can adapt the active forwarding topology after link-state changes.

5.1 Core Switch-Side Configuration

```
ovs-vsctl set bridge br0 protocols=OpenFlow13
ovs-vsctl set-controller br0 tcp:10.0.0.10:6653
ovs-vsctl set-fail-mode br0 secure
ovs-vsctl list controller
ovs-ofctl -O OpenFlow13 dump-flows br0
```

All four OVS switches were verified with is_connected=true and state=ACTIVE before host testing. Controller-installed OpenFlow rules were also visible in the br0 flow tables.

5.2 FAUCET Configuration Logic

```
vlans:
  users:
    vid: 100

dps:
  OVS1:
    dp_id: 0x0000e2a24fcdff4a
    stack:
      priority: 1
    interfaces:
      2: {native_vlan: users}      # H1
      3: {native_vlan: users}      # H2
      4: {native_vlan: users}      # H3
      5:
        stack: {dp: OVS2, port: 2}
      6:
        stack: {dp: OVS4, port: 2}
# OVS2, OVS3 and OVS4 contain reciprocal stack-link definitions.
```

The complete implementation used the actual datapath IDs and reciprocal port mappings listed in Table 2. The configuration was checked with check_faucet_config before reloading FAUCET. Stack ports were not configured as ordinary native-VLAN access ports, which avoids treating the redundant diamond as an uncontrolled Layer-2 loop.

6. Experimental Methodology

6.1 Test Environment

The implementation was built in GNS3 using lightweight Linux/Alpine-based hosts, four Open vSwitch instances, a FAUCET controller container, and a GNS3 Ethernet switch for management connectivity. OVS management interfaces use eth0, while data-plane interfaces use eth1 and above. The controller listens on TCP port 6653. All application hosts use VLAN 100 and the 192.168.10.0/24 subnet.

6.2 Failure Event

The required changing network condition is a deliberate failure of the OVS2-to-OVS3 link. The failure is injected by administratively setting OVS2 eth2 down. In the baseline, the OVS4 path is also disabled, so the network cannot recover automatically. In the proposed SDN condition, the OVS4 path remains available and FAUCET stacking can restore forwarding through OVS1 → OVS4 → OVS3.

6.3 Metrics

Metric	Measurement method	Interpretation
ICMP packet loss	ping -i 0.2	Packets not delivered during failure/recovery
RTT	ping summary	Latency for successfully delivered ICMP packets
Recovery interval	Missing ping sequence count × 0.2 s	Approximate network connectivity interruption
Throughput	iperf3 interval and summary output	Application data rate
UDP loss / jitter	iperf3 UDP receiver summary	Video-service degradation
TCP retransmissions	iperf3 TCP summary	Transport recovery effort after interruption
Path usage	ovs-ofctl dump-ports counters	Evidence that traffic moves across alternate link

6.4 Experimental Procedure

Step	Procedure
1	Verify all OVS switches are connected to FAUCET and normal H1-to-Server ping succeeds.
2	Collect reference traffic and port counters under normal operation.
3	Baseline: disable OVS1 eth5 (backup path), run continuous ping, then fail OVS2 eth2.
4	Restore the failed baseline links and verify connectivity.
5	Proposed SDN: keep both paths available, run continuous ping, then fail the active OVS2-to-OVS3 link.
6	Verify automatic recovery and check OVS4 port counters to confirm alternate-path forwarding.
7	Run the three 60 s iperf3 services and inject the same link failure during active traffic.
8	Record receiver throughput, loss, jitter, TCP retransmissions, and outage behavior.

```
Server: iperf3 -s -p 5201 -D
        iperf3 -s -p 5202 -D
        iperf3 -s -p 5203 -D

H1:iperf3 -c 192.168.10.100 -p 5201 -u -b 6M -t 60 -i 1
H2:iperf3 -c 192.168.10.100 -p 5202 -b 3M -t 60 -i 1
H3:iperf3 -c 192.168.10.100 -p 5203 -b 8M -t 60 -i 1
```

7. Experimental Results

7.1 Normal Operation

Before failure injection, H1 could reach the server successfully. A five-packet reference ping recorded 0% loss with a mean RTT of 19.246 ms. A later longer stable run in the restored redundant topology recorded 133/133 replies, 0% packet loss, and a mean RTT of 11.652 ms. These tests confirmed that the hosts, VLAN, controller, and datapath forwarding were working before the failure experiments.

7.2 Baseline Single-Path Failure

With OVS1 eth5 disabled, only the OVS1 → OVS2 → OVS3 path remained. After OVS2 eth2 was set down, the continuous ping produced “Destination Host Unreachable” messages and did not recover through an alternate path. The captured run transmitted 390 probes and received 215, producing 44.8718% packet loss. Connectivity returned only after manual restoration. Therefore, the baseline result is best described as “no automatic recovery” rather than assigning a convergence time, because the outage duration depended on when the operator restored the link.

7.3 SDN Redundant Failover

With both stack paths available, the same class of OVS2-to-OVS3 failure caused a temporary interruption, after which ICMP replies resumed automatically. The earlier full-length measured run transmitted 293 probes and received 229, for 21.843% packet loss. In the repeat evidence capture, replies were observed through sequence 18 and resumed at sequence 85, implying 66 missed 0.2 s probes. The repeat-run connectivity interruption is therefore $66 \times 0.2 \approx 13.2$ s. Unlike the baseline, no manual path reconfiguration was required to restore end-to-end reachability.

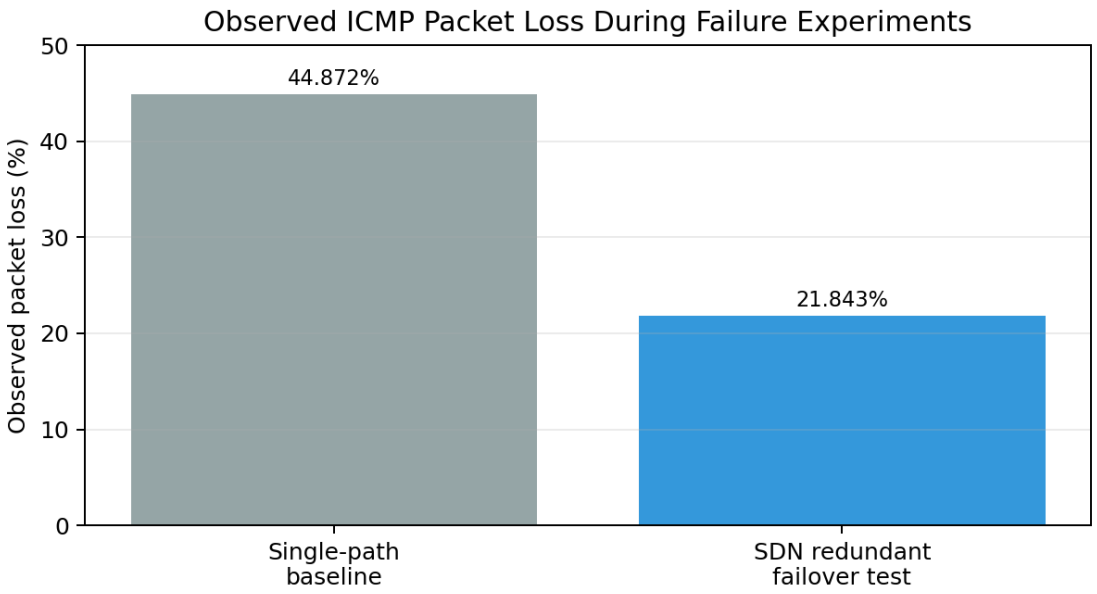


Figure 2. Observed packet loss in the two recorded failure runs. The run durations and manual restoration timing were different, so the percentages should be interpreted as observed test outcomes rather than a controlled percentage-improvement claim.

7.4 Before-vs-After Summary

Measure	Baseline: single path	Proposed: redundant SDN
Normal connectivity	Working	Working
Failure injected	OVS2 → OVS3 link down	OVS2 → OVS3 link down
Alternate path available	No	Yes, via OVS4
Observed ICMP loss	44.8718%	21.843%
Automatic recovery	No	Yes
Approx. ICMP recovery interval	Not applicable; manual restore required	≈ 13.2 s
Operational behavior	Service remains unavailable until operator action	Forwarding resumes through redundant SDN topology

7.5 Simultaneous Three-Service Failure Test

The three application flows were run for 60 s while the redundant SDN network experienced the link failure. The final iperf3 summaries show that UDP and TCP reacted differently. UDP packets transmitted during the interruption were not recovered, so the video receiver recorded reduced throughput and packet loss. The TCP flows experienced long zero-throughput intervals but later retransmitted outstanding data and returned to their configured rates after connectivity recovered.

Service	Offered / target	Measured summary	Failure indicators
H1 Video (UDP)	6.00 Mbit/s	Receiver 4.41 Mbit/s; 31.5 MB received	1.602 ms jitter; 8248/31078 datagrams lost = 27%
H2 Business (TCP)	3.00 Mbit/s	3.01 Mbit/s average; 21.5 MB	103 retransmissions; ≈26 s zero-throughput interval
H3 File (TCP)	8.00 Mbit/s	8.00 Mbit/s average; 57.2 MB	177 retransmissions; ≈27 s zero-throughput interval

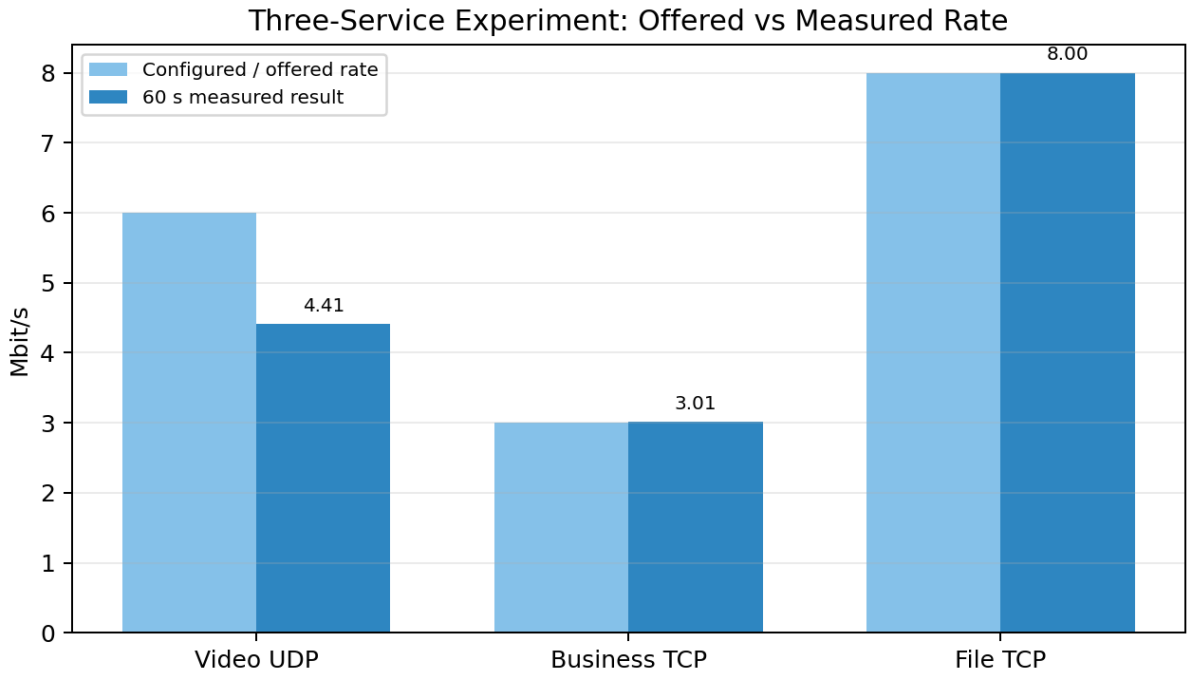


Figure 3. Configured/offered service rates compared with 60 s measured summaries. TCP averages include post-recovery catch-up traffic; UDP receiver throughput reflects unrecovered datagram loss.

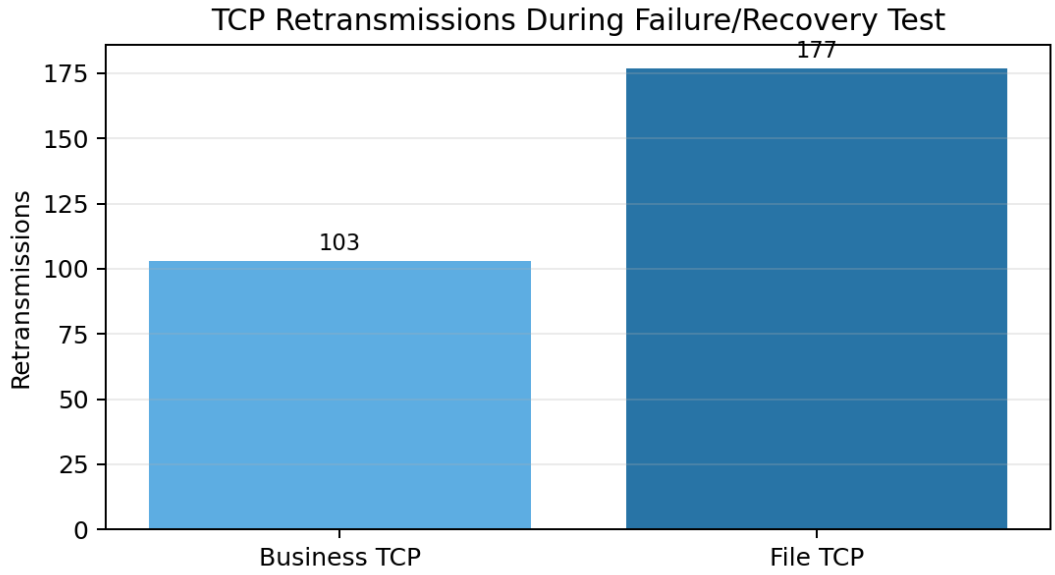


Figure 4. TCP retransmissions recorded during the failure/recovery test.

The TCP averages should not be interpreted as continuous service. H2 showed approximately 26 s at 0 bit/s, followed by short 34.6 and 49.3 Mbit/s catch-up intervals before returning to about 3 Mbit/s. H3 showed approximately 27 s at 0 bit/s, followed by recovery bursts up to about 65.1 Mbit/s before returning to about 8 Mbit/s. These bursts are consistent with queued/retransmitted TCP data being delivered after the path became usable again. In contrast, the UDP sender remained at 6 Mbit/s but the receiver averaged 4.41 Mbit/s and lost 27% of datagrams because UDP does not provide TCP-style retransmission.

7.6 Application-Level Disruption

The business TCP flow showed approximately 26 seconds of zero throughput, while the file-transfer TCP flow showed approximately 27 seconds. After the network path recovered, both TCP streams produced short high-rate bursts as previously unacknowledged data was delivered and the connections recovered

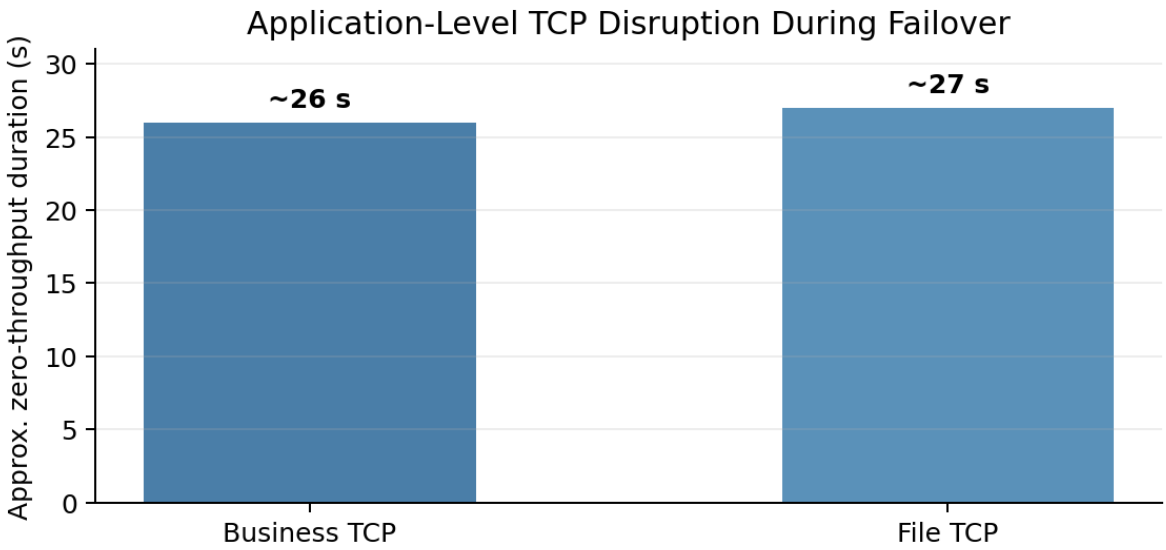


Figure 5. Approximate application-level zero-throughput interval observed for the two TCP services.

7.7 Alternate Path Verification

The rerouting conclusion was also checked at the switch level. While the failed primary link was unavailable, OVS4 eth2 (OpenFlow port 3, toward OVS3) counters increased from 515,590 to 528,567 received packets and from 599,887 to 655,766 transmitted packets. This is an increase of 12,977 received packets and 55,879 transmitted packets. Byte counters increased by 909,847 bytes received and 84,822,891 bytes transmitted. The counter growth while the OVS2-to-OVS3 path was unavailable provides direct evidence that traffic was traversing the OVS4 alternate path.

Counter	Earlier value	Later value	Increase
RX packets	515,590	528,567	12,977
TX packets	599,887	655,766	55,879
RX bytes	43,241,261	44,151,108	909,847
TX bytes	199,329,789	284,152,680	84,822,891

8. Discussion

8.1 Does the Proposed Solution Improve Network Behaviour?

Yes, in the specific sense tested by this project: the redundant SDN topology restores end-to-end forwarding after the active transit link fails, whereas the single-path baseline does not. The most important improvement is therefore operational resilience, not a claim of permanently higher throughput. In the baseline, a critical link failure removes the only end-to-end path and service remains unavailable until an operator restores the link. With FAUCET stacking and the alternate OVS4 path enabled, the network eventually resumes forwarding without manual path configuration.

The ICMP experiments also show that recovery is not instantaneous. The repeat evidence run showed an estimated network-level interruption of about 13.2 s. The application experiment showed even longer zero-throughput intervals for the TCP flows. These results are important because they prevent an exaggerated conclusion: SDN improves the ability to recover automatically, but the measured controller/topology detection and transport recovery still produce a noticeable service interruption.

8.2 Why UDP and TCP Results Differ

The UDP video flow lost 27% of datagrams and the receiver averaged 4.41 Mbit/s even though the sender continued transmitting at 6 Mbit/s. This is expected for an unreliable datagram service because packets lost during the outage are not retransmitted by UDP **[6].** TCP behaves differently: the business and file flows temporarily fall to zero, then retransmit and catch up after the path is restored. Their 60 s average rates are therefore close to the configured values even though the user would still experience a significant mid-test interruption. RFC 9293 provides the standards basis for TCP reliable delivery and retransmission behavior **[5].**

8.3 Relation to SDN Architecture

The experiment demonstrates the separation of control and forwarding in a practical way. The OVS switches forward data, while the FAUCET controller programs the network behavior through OpenFlow. The alternate link is not treated as an uncontrolled forwarding loop; it is described to FAUCET as part of the stack topology. The result aligns with ONF SDN architecture principles and with FAUCET's documented use of stacking for resilient multi-switch topologies **[1], [3].**

9. Limitations and Validity

This is a virtual GNS3 laboratory rather than a hardware campus deployment, so absolute latency and failover values are influenced by host CPU scheduling, Docker/virtualization overhead, and the emulated environment. The baseline and SDN ICMP tests were not identical in total duration because baseline restoration was manual; consequently, their packet-loss percentages should be reported as observed outcomes, not as a precise percentage improvement. The three iperf3 clients were launched manually and were not time-synchronized, so the 26-27 s application outage windows are approximate. Only one failure type, the OVS2-to-OVS3 link failure, was evaluated. Future work could automate timestamped failure injection, repeat each test multiple times, evaluate controller redundancy, and study congestion-aware path selection or QoS.

9.1 Evidence

Controller evidence:

```
ovs-vsctl list controller
ovs-ofctl -O OpenFlow13 dump-flows br0 | head -n 15
```

```
OVS1:/# ovs-vsctl list controller
_uuid               : 98ea18a9-285a-4b14-bc55-70b731a2d919
connection_mode     : []
controller_burst_limit: []
controller_queue_size: []
controller_rate_limit: []
enable_async_messages: []
external_ids        : {}
inactivity_probe    : []
is_connected        : true
local_gateway       : []
local_ip            : []
local_netmask       : []
max_backoff         : []
other_config        : {}
role                : other
status              : {last_error="Broken pipe", sec_since_connect="6721", sec_since_disconnect="6722", state=ACTIVE}
target              : "tcp:10.0.0.10:6653"
type                : []
OVS1:/# ovs-ofctl -O OpenFlow13 dump-flows br0 | head -n 15
OFPST_FLOW reply (OF1.3) (xid=0x2):
 cookie=0x5adc15c0, duration=6741.686s, table=0, n_packets=1122, n_bytes=97614, priority=9099,in_port=5,dl_dst=01:80:c2
:00:00:00/ff:ff:ff:ff:ff:f0,dl_type=0x88cc actions=CONTROLLER:128
 cookie=0x5adc15c0, duration=6741.686s, table=0, n_packets=1122, n_bytes=97614, priority=9099,in_port=6,dl_dst=01:80:c2
:00:00:00/ff:ff:ff:ff:ff:f0,dl_type=0x88cc actions=CONTROLLER:128
 cookie=0x5adc15c0, duration=6741.686s, table=0, n_packets=1, n_bytes=70, priority=4097,in_port=5,vlan_tci=0x0000/0x1ff
f actions=drop
 cookie=0x5adc15c0, duration=6741.686s, table=0, n_packets=1, n_bytes=70, priority=4097,in_port=6,vlan_tci=0x0000/0x1ff
f actions=drop
 cookie=0x5adc15c0, duration=6741.686s, table=0, n_packets=1, n_bytes=70, priority=4096,in_port=2,vlan_tci=0x0000/0x1ff
f actions=push_vlan:0x8100,set_field:4196->vlan_vid,goto_table:1
 cookie=0x5adc15c0, duration=6741.686s, table=0, n_packets=1, n_bytes=70, priority=4096,in_port=3,vlan_tci=0x0000/0x1ff
f actions=push_vlan:0x8100,set_field:4196->vlan_vid,goto_table:1
 cookie=0x5adc15c0, duration=6741.686s, table=0, n_packets=2, n_bytes=140, priority=4096,in_port=4,vlan_tci=0x0000/0x1f
ff actions=push_vlan:0x8100,set_field:4196->vlan_vid,goto_table:1
 cookie=0x5adc15c0, duration=6741.686s, table=0, n_packets=2, n_bytes=148, priority=4096,in_port=5 actions=goto_table:1
 cookie=0x5adc15c0, duration=6741.686s, table=0, n_packets=0, n_bytes=0, priority=4096,in_port=6 actions=goto_table:1
 cookie=0x5adc15c0, duration=6741.686s, table=0, n_packets=2, n_bytes=140, priority=0 actions=drop
```

Figure 6. Open vSwitch connected to the FAUCET SDN controller through OpenFlow 1.3, with controller-installed forwarding rules.

Baseline failure evidence:

```
H1:ping -i 0.2 192.168.10.100
OVS1:ip link set eth5 down
OVS2:ip link set eth2 down
```

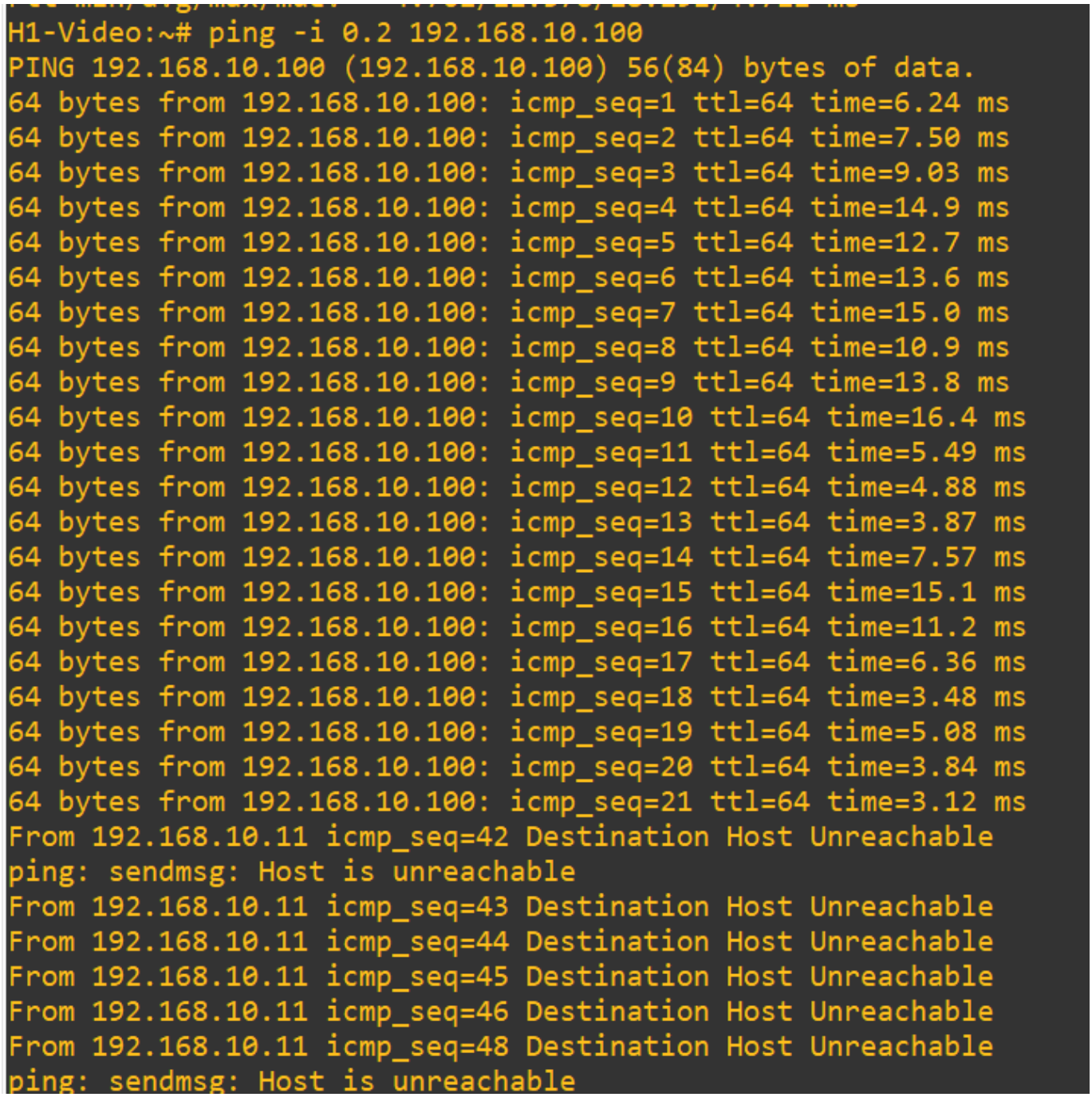


Figure 7. Baseline single-path failure. With the backup path disabled, failure of the OVS2-OVS3 link caused loss of connectivity and no automatic recovery.

SDN automatic recovery evidence

```
H1:ping -i 0.2 192.168.10.100
OVS2:ip link set eth2 down
```

```
H1-Video:~# ping -i 0.2 192.168.10.100
PING 192.168.10.100 (192.168.10.100) 56(84) bytes of data.
64 bytes from 192.168.10.100: icmp_seq=1 ttl=64 time=24.6 ms
64 bytes from 192.168.10.100: icmp_seq=2 ttl=64 time=12.0 ms
64 bytes from 192.168.10.100: icmp_seq=3 ttl=64 time=16.9 ms
64 bytes from 192.168.10.100: icmp_seq=4 ttl=64 time=11.2 ms
64 bytes from 192.168.10.100: icmp_seq=5 ttl=64 time=9.62 ms
64 bytes from 192.168.10.100: icmp_seq=6 ttl=64 time=12.2 ms
64 bytes from 192.168.10.100: icmp_seq=7 ttl=64 time=8.12 ms
64 bytes from 192.168.10.100: icmp_seq=8 ttl=64 time=10.9 ms
64 bytes from 192.168.10.100: icmp_seq=9 ttl=64 time=10.3 ms
64 bytes from 192.168.10.100: icmp_seq=10 ttl=64 time=13.1 ms
64 bytes from 192.168.10.100: icmp_seq=11 ttl=64 time=9.86 ms
64 bytes from 192.168.10.100: icmp_seq=12 ttl=64 time=13.1 ms
64 bytes from 192.168.10.100: icmp_seq=13 ttl=64 time=14.4 ms
64 bytes from 192.168.10.100: icmp_seq=14 ttl=64 time=14.5 ms
64 bytes from 192.168.10.100: icmp_seq=15 ttl=64 time=17.5 ms
64 bytes from 192.168.10.100: icmp_seq=16 ttl=64 time=10.1 ms
64 bytes from 192.168.10.100: icmp_seq=17 ttl=64 time=7.32 ms
64 bytes from 192.168.10.100: icmp_seq=18 ttl=64 time=19.8 ms
64 bytes from 192.168.10.100: icmp_seq=85 ttl=64 time=27.9 ms
64 bytes from 192.168.10.100: icmp_seq=86 ttl=64 time=10.9 ms
64 bytes from 192.168.10.100: icmp_seq=87 ttl=64 time=7.50 ms
64 bytes from 192.168.10.100: icmp_seq=88 ttl=64 time=8.18 ms
64 bytes from 192.168.10.100: icmp_seq=89 ttl=64 time=8.12 ms
64 bytes from 192.168.10.100: icmp_seq=90 ttl=64 time=8.62 ms
64 bytes from 192.168.10.100: icmp_seq=91 ttl=64 time=9.57 ms
64 bytes from 192.168.10.100: icmp_seq=92 ttl=64 time=8.69 ms
64 bytes from 192.168.10.100: icmp_seq=93 ttl=64 time=6.18 ms
64 bytes from 192.168.10.100: icmp_seq=94 ttl=64 time=8.62 ms
64 bytes from 192.168.10.100: icmp_seq=95 ttl=64 time=13.5 ms
64 bytes from 192.168.10.100: icmp_seq=96 ttl=64 time=10.3 ms
64 bytes from 192.168.10.100: icmp_seq=97 ttl=64 time=14.0 ms
64 bytes from 192.168.10.100: icmp_seq=98 ttl=64 time=9.22 ms
64 bytes from 192.168.10.100: icmp_seq=99 ttl=64 time=3.31 ms
64 bytes from 192.168.10.100: icmp_seq=100 ttl=64 time=11.6 ms
64 bytes from 192.168.10.100: icmp_seq=101 ttl=64 time=10.7 ms
64 bytes from 192.168.10.100: icmp_seq=102 ttl=64 time=11.1 ms
```

Figure 8. SDN-controlled recovery in the repeat evidence run: replies resumed after approximately 13.2 s, confirming automatic forwarding through the redundant path.

Prove traffic actually uses OVS4

```
OVS4:echo "FIRST READING"; date; ovs-ofctl -O OpenFlow13 dump-ports br0 3; sleep 5; echo "SECOND READING"; date;
ovs-ofctl -O OpenFlow13 dump-ports br0 3
OVS2:ip link set eth2 down
H1:iperf3 -c 192.168.10.100 -t 20
```

```
OVS4:/# echo "FIRST READING"; date; ovs-ofctl -O OpenFlow13 dump-ports br0 3; sleep 5; echo "SECOND READING"; date; ovs
-ofctl -O OpenFlow13 dump-ports br0 3
FIRST READING
Tue Sep 22 22:09:52 UTC 2026
OFPST_PORT reply (OF1.3) (xid=0x2): 1 ports
  port  eth2: rx pkts=515590, bytes=43241261, drop=1, errs=0, frame=0, over=0, crc=0
           tx pkts=599887, bytes=199329789, drop=0, errs=0, coll=0
           duration=23205.044s
SECOND READING
Tue Sep 22 22:09:57 UTC 2026
OFPST_PORT reply (OF1.3) (xid=0x2): 1 ports
  port  eth2: rx pkts=528567, bytes=44151108, drop=1, errs=0, frame=0, over=0, crc=0
           tx pkts=655766, bytes=284152680, drop=0, errs=0, coll=0
           duration=23210.133s
```

Figure 9. OVS4 port-counter growth after failure of the primary OVS2-OVS3 link, confirming that traffic was redirected through the alternate OVS1-OVS4-OVS3 path.

10. Conclusion

This project implemented and evaluated an SDN-controlled redundant campus network using FAUCET, Open vSwitch, OpenFlow 1.3, and GNS3. The network included more than the required six devices and three separate services: UDP video, TCP business traffic, and TCP file transfer. A basic single-path baseline was compared with a redundant FAUCET stacking configuration under the same OVS2-to-OVS3 link-failure scenario. The baseline lost connectivity and required manual restoration. In contrast, the SDN topology recovered automatically through the alternate OVS4 path. A repeat SDN ICMP evidence run recovered after an estimated 13.2 s interruption, and OVS4 port counters confirmed alternate-path traffic. The three-flow experiment quantified the service impact: UDP video received 4.41 Mbit/s with 27% loss and 1.602 ms jitter, while the TCP business and file flows recorded 103 and 177 retransmissions respectively and resumed after temporary outages. These results support the conclusion that the proposed SDN design improves resilience and operational adaptability, although failover delay remains a limitation that should be optimized in future work.

References

[1] Open Networking Foundation (ONF), “Software-Defined Networking (SDN) Definition.” https://opennetworking.org/sdn-definition/ (accessed 2026).

[2] Open Networking Foundation (ONF), “OpenFlow Switch Specification, Version 1.3.5,” Apr. 2015. Available through ONF Specifications / Open Datapath resources.

[3] FAUCET, “Stacking tutorial,” FAUCET Documentation. https://docs.faucet.nz/en/1.9.41/tutorials/stacking.html (accessed 2026).

[4] Open vSwitch Project, “Open vSwitch Advanced Features,” Open vSwitch Documentation. https://docs.openvswitch.org/en/latest/tutorials/ovs-advanced/ (accessed 2026).

[5] W. Eddy, Ed., “Transmission Control Protocol (TCP),” RFC 9293, IETF, Aug. 2022. https://www.rfc-editor.org/rfc/rfc9293.html

[6] J. Postel, “User Datagram Protocol,” RFC 768, Aug. 1980. https://www.rfc-editor.org/rfc/rfc768.html

[7] ESnet / Lawrence Berkeley National Laboratory, “iperf3 Documentation - Invoking iperf3.” https://software.es.net/iperf/invoking.html (accessed 2026).